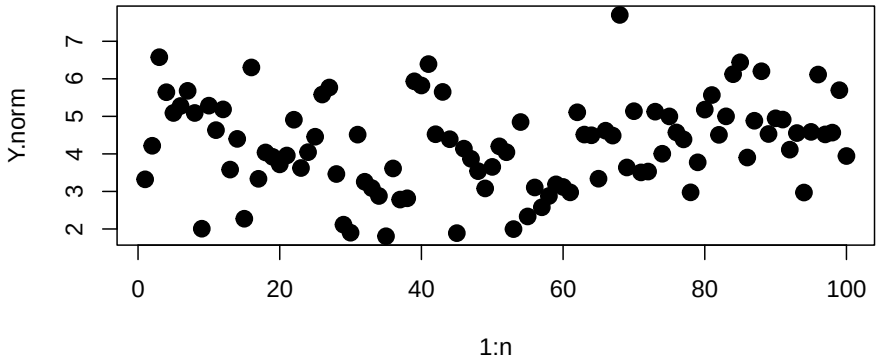
Revised Lecture 23 Code: Hidden Markov Models

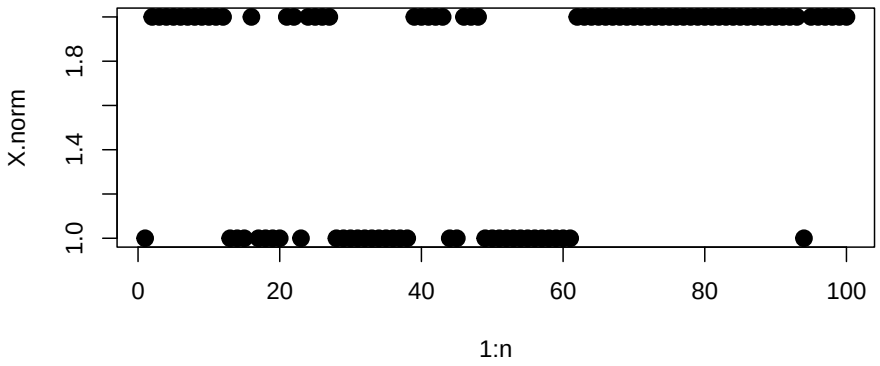
2026-04-10

Data

Data is read in from a pre-simulated 2-state HMM with Normal emissions and true parameters $\mu = (3, 5)$, $\nu_{11} = \nu_{22} = 0.9$, and $\pi = (0.5, 0.5)$. Global constants $n = 100$ observations and $m = 2$ states are defined here. The observed data Y and true hidden states X are plotted side by side.

```
################################################
#### READING IN DATA GENERATED FROM HMM WITH ####
#### NORMAL NOISE: mu=(3,5),nu[1,1] = 0.9    ####
################################################

data <- read.table("data/hmm.norm.txt", header = F)
Y.norm <- data[, 1]
X.norm <- data[, 2]

truemu <- c(3, 5)
truenu <- rbind(c(0.9, 0.1), c(0.1, 0.9))
truepi <- c(0.5, 0.5)

n <- 100
m <- 2

par(mfrow = c(2, 1))
plot(1:n, Y.norm, pch = 19, cex = 1.5)
plot(1:n, X.norm, pch = 19, cex = 1.5)
```



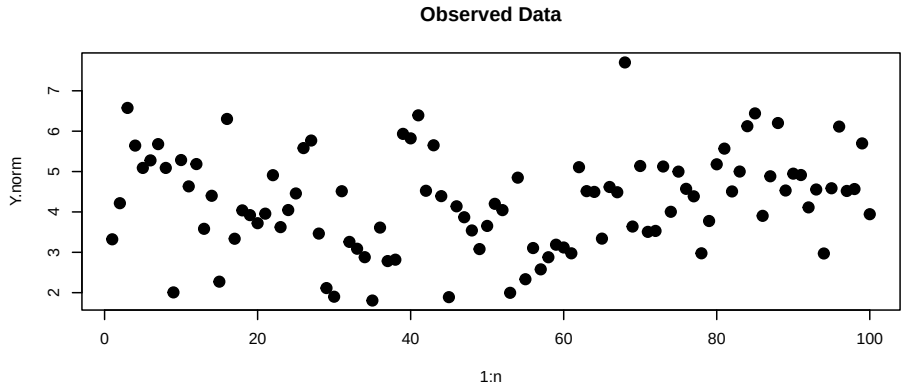


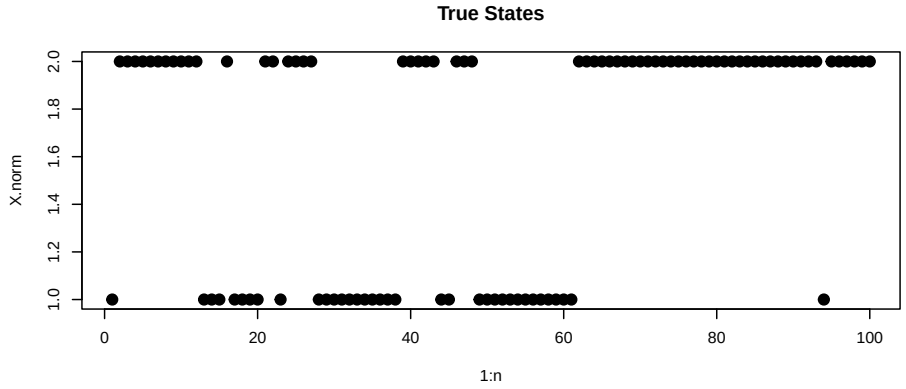
Viterbi Algorithm

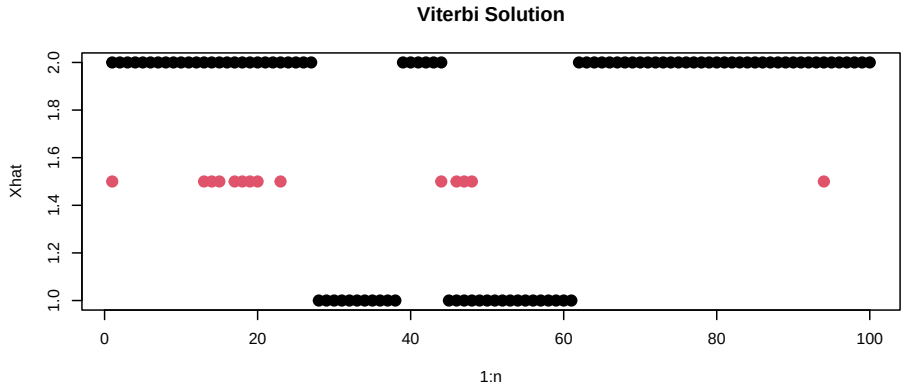
The Viterbi algorithm finds the single most probable sequence of hidden states $\hat{X}_{1:n}$ given the observed data and true parameters. It runs in two passes: a forward pass fills in $\delta_{t,i}$, the probability of the most likely path ending in state i at time t; a backward trace then recovers the optimal state sequence. The inferred states are compared against the true states, with mismatches highlighted in red.

```
################################################
####### VITERBI ALGORITHM FOR NORMAL DATA #######
################################################

norm.density <- function(obs, state, mu) {
    if (state == 1) {
        out <- dnorm(obs, mean = mu[1], sd = 1)
    }
    if (state == 2) {
        out <- dnorm(obs, mean = mu[2], sd = 1)
    }
    out
}
```

```r
delta <- matrix(NA, nrow = n, ncol = m)
## Initialization:
for (i in 1:m) {
    delta[1, i] <- truepi[i] * norm.density(Y.norm[1], i, truemu)
}
## Recursion:
for (t in 2:n) {
    for (i in 1:m) {
        temp <- rep(NA, m)
        for (j in 1:m) {
            temp[j] <- delta[t - 1, j] * truenu[j, i] * norm.density(Y.norm[t], i, truemu)
        }
        delta[t, i] <- max(temp)
    }
}
## Tracing Back:
Xhat <- rep(NA, n)
Xhat[n] <- which(delta[n, ] == max(delta[n, ]))
for (t in (n - 1):1) {
    temp <- rep(NA, m)
    for (j in 1:m) {
        temp[j] <- delta[t, j] * truenu[j, Xhat[t + 1]]
    }
    Xhat[t] <- which(temp == max(temp))
}

### checking our inferred states
par(mfrow = c(3, 1))
plot(1:n, Y.norm, pch = 19, main = "Observed Data", cex = 1.5)
plot(1:n, X.norm, pch = 19, main = "True States", cex = 1.5)
plot(1:n, Xhat, pch = 19, main = "Viterbi Solution", cex = 1.5)
for (i in 1:n) {
    if (Xhat[i] != X.norm[i]) {
        points(i, 1.5, col = 2, pch = 19, cex = 1.5)
    }
}
```







Gibbs Sampler: Full Posterior Inference

Instead of finding a single best state sequence, the Gibbs sampler draws from the full joint posterior $p(X_{1:n}, \nu, \pi, \mu \mid Y_{1:n})$. Each iteration has three steps:

1. **Step A (FFBS):** Sample the hidden state sequence $X_{1:n}$ using Forward-Filtering Backward-Sampling.
2. **Step B:** Sample each row of the transition matrix ν and the initial state distribution π from Dirichlet full conditionals (using the Gamma trick).
3. **Step C:** Sample each emission mean μ_k from its Normal full conditional, given the observations currently assigned to state k.

Two independent chains are run from different starting values to assess convergence.

```r
################################################
#### FULL POSTERIOR SAMPLING FOR NORMAL DATA ####
################################################

numsamp <- 10000
X.samp <- matrix(NA, nrow = numsamp, ncol = n)
nu.samp <- matrix(NA, nrow = numsamp, ncol = m * m)
pi.samp <- matrix(NA, nrow = numsamp, ncol = m)
mu.samp <- matrix(NA, nrow = numsamp, ncol = m)

curnu <- rbind(c(0.75, 0.25), c(0.25, 0.75))
curpi <- c(0.5, 0.5)
curmu <- c(2, 4)
curX <- rep(NA, n)

for (iter in 1:numsamp) {
    #### Forward Algorithm ####
    alpha <- matrix(NA, nrow = n, ncol = m)
    for (i in 1:m) {
        alpha[1, i] <- curpi[i] * norm.density(Y.norm[1], i, curmu)
    }
    for (t in 2:n) {
        for (i in 1:m) {
            temp <- rep(NA, m)
            for (j in 1:m) {
                temp[j] <- alpha[t - 1, j] * curnu[j, i] * norm.density(Y.norm[t], i, curmu)
            }
            alpha[t, i] <- sum(temp)
        }
    }
    #### Backwards Sampling ####
    probvec <- alpha[n, ] / sum(alpha[n, ])
    curX[n] <- sample(1:m, size = 1, prob = probvec)
    for (t in (n - 1):1) {
        probvec <- rep(NA, m)
        for (i in 1:m) {
            probvec[i] <- alpha[t, i] * curnu[i, curX[t + 1]]
        }
        probvec <- probvec / sum(probvec)
        curX[t] <- sample(1:m, size = 1, prob = probvec)
    }
    #### Calculating N matrix ####
    Nmat <- matrix(NA, nrow = m, ncol = m)
    for (j in 1:m) {
        for (k in 1:m) {
            Nmat[j, k] <- 0
            for (t in 2:n) {
                if (curX[t - 1] == j && curX[t] == k) {
                    Nmat[j, k] <- Nmat[j, k] + 1
                }
            }
        }
    }
```

```
    #### Sampling Nu rows from Dirichlet (via Gammas) ####
    for (j in 1:m) {
        for (k in 1:m) {
            curnu[j, k] <- rgamma(1, shape = (Nmat[j, k] + 1), rate = 1)
        }
        curnu[j, ] <- curnu[j, ] / sum(curnu[j, ])
    }
    #### Calculating N vector ####
    Nvec <- rep(NA, m)
    for (j in 1:m) {
        Nvec[j] <- sum(curX == j)
    }
    #### Sampling pi from Dirichlet (via Gammas) ####
    for (j in 1:m) {
        curpi[j] <- rgamma(1, shape = (Nvec[j] + 1), rate = 1)
    }
    curpi <- curpi / sum(curpi)
    #### Sampling means of emission model
    # for (j in 1:m){
    #   curmean <- mean(Y.norm[curX==j])
    #   curvar <- 1/sum(curX==j)
    #   curmu[j] <- rnorm(1,curmean,sqrt(curvar))
    # }
    ## -- C1: SAMPLE EMISSION MEANS (FIXED) --
    for (j in 1:m) {
        n_obs_in_state <- sum(curX == j)

        if (n_obs_in_state > 0) {
            # Standard update if the state has at least one observation
            curmean <- mean(Y.norm[curX == j])
            curvar <- 1 / n_obs_in_state
            curmu[j] <- rnorm(1, curmean, sqrt(curvar))
        } else {
            # FIX: If the state is completely empty, we have no data to update it.
            # Keep the mean from the previous iteration to prevent NAs.
            curmu[j] <- curmu[j]
        }
    }
    #### Storing parameter values
    X.samp[iter, ] <- curX
    nu.samp[iter, ] <- as.vector(curnu)
    pi.samp[iter, ] <- curpi
    mu.samp[iter, ] <- curmu
    # print(iter)
}

X.samp1 <- X.samp
nu.samp1 <- nu.samp
pi.samp1 <- pi.samp
mu.samp1 <- mu.samp

### re-running with different starting values
```

```r
curnu <- rbind(c(0.5, 0.5), c(0.5, 0.5))
curpi <- c(0.25, 0.75)
curmu <- c(1, 5)
curX <- rep(NA, n)

for (iter in 1:numsamp) {
    #### Forward Algorithm ####
    alpha <- matrix(NA, nrow = n, ncol = m)
    for (i in 1:m) {
        alpha[1, i] <- curpi[i] * norm.density(Y.norm[1], i, curmu)
    }
    for (t in 2:n) {
        for (i in 1:m) {
            temp <- rep(NA, m)
            for (j in 1:m) {
                temp[j] <- alpha[t - 1, j] * curnu[j, i] * norm.density(Y.norm[t], i, curmu)
            }
            alpha[t, i] <- sum(temp)
        }
    }
    #### Backwards Sampling ####
    probvec <- alpha[n, ] / sum(alpha[n, ])
    curX[n] <- sample(1:m, size = 1, prob = probvec)
    for (t in (n - 1):1) {
        probvec <- rep(NA, m)
        for (i in 1:m) {
            probvec[i] <- alpha[t, i] * curnu[i, curX[t + 1]]
        }
        probvec <- probvec / sum(probvec)
        curX[t] <- sample(1:m, size = 1, prob = probvec)
    }
    #### Calculating N matrix ####
    Nmat <- matrix(NA, nrow = m, ncol = m)
    for (j in 1:m) {
        for (k in 1:m) {
            Nmat[j, k] <- 0
            for (t in 2:n) {
                if (curX[t - 1] == j && curX[t] == k) {
                    Nmat[j, k] <- Nmat[j, k] + 1
                }
            }
        }
    }
    #### Sampling Nu rows from Dirichlet (via Gammas) ####
    for (j in 1:m) {
        for (k in 1:m) {
            curnu[j, k] <- rgamma(1, shape = (Nmat[j, k] + 1), rate = 1)
        }
        curnu[j, ] <- curnu[j, ] / sum(curnu[j, ])
    }
    #### Calculating N vector ####
    Nvec <- rep(NA, m)
    for (j in 1:m) {
```

```
        Nvec[j] <- sum(curX == j)
    }
    #### Sampling pi from Dirichlet (via Gammas) ####
    for (j in 1:m) {
        curpi[j] <- rgamma(1, shape = (Nvec[j] + 1), rate = 1)
    }
    curpi <- curpi / sum(curpi)
    #### Sampling means of emission model
    # for (j in 1:m){
    #    curmean <- mean(Y.norm[curX==j])
    #    curvar <- 1/sum(curX==j)
    #    curmu[j] <- rnorm(1,curmean,sqrt(curvar))
    # }

    ## -- C1: SAMPLE EMISSION MEANS (FIXED) --
    for (j in 1:m) {
        n_obs_in_state <- sum(curX == j)

        if (n_obs_in_state > 0) {
            # Standard update if the state has at least one observation
            curmean <- mean(Y.norm[curX == j])
            curvar <- 1 / n_obs_in_state
            curmu[j] <- rnorm(1, curmean, sqrt(curvar))
        } else {
            # FIX: If the state is completely empty, we have no data to update it.
            # Keep the mean from the previous iteration to prevent NAs.
            curmu[j] <- curmu[j]
        }
    }
    #### Storing parameter values
    X.samp[iter, ] <- curX
    nu.samp[iter, ] <- as.vector(curnu)
    pi.samp[iter, ] <- curpi
    mu.samp[iter, ] <- curmu
    # print(iter)
}

X.samp2 <- X.samp
nu.samp2 <- nu.samp
pi.samp2 <- pi.samp
mu.samp2 <- mu.samp
```

Label Switching Correction

The HMM likelihood is symmetric in the state labels: swapping all 1s and 2s produces an identical likelihood. As a result, the sampler can arbitrarily re-assign which numeric label corresponds to which physical state across iterations. This is called *label switching*. It is corrected post-hoc by enforcing the identifiability constraint $\mu_1 < \mu_2$: any iteration where $\mu_1 > \mu_2$ has its labels swapped for μ, π, ν, and X.

```
### 1. DEFINE A REUSABLE FUNCTION TO FIX LABEL SWITCHING ###
fix_label_switching <- function(mu_mat, nu_mat, pi_mat, X_mat) {
    n_iter <- nrow(mu_mat)
```

```
    for (i in 1:n_iter) {
        if (mu_mat[i, 1] > mu_mat[i, 2]) {
            # Swap mu
            temp_mu <- mu_mat[i, 1]
            mu_mat[i, 1] <- mu_mat[i, 2]
            mu_mat[i, 2] <- temp_mu

            # Swap pi
            temp_pi <- pi_mat[i, 1]
            pi_mat[i, 1] <- pi_mat[i, 2]
            pi_mat[i, 2] <- temp_pi

            # Swap nu
            temp_nu11 <- nu_mat[i, 1]
            temp_nu21 <- nu_mat[i, 2]
            temp_nu12 <- nu_mat[i, 3]
            temp_nu22 <- nu_mat[i, 4]

            nu_mat[i, 1] <- temp_nu22 # New nu11
            nu_mat[i, 4] <- temp_nu11 # New nu22
            nu_mat[i, 2] <- temp_nu12 # New nu21
            nu_mat[i, 3] <- temp_nu21 # New nu12

            # Swap X
            current_X <- X_mat[i, ]
            X_mat[i, current_X == 1] <- 2
            X_mat[i, current_X == 2] <- 1
        }
    }
    # Return the corrected matrices as a list
    return(list(mu = mu_mat, nu = nu_mat, pi = pi_mat, X = X_mat))
}

### 2. APPLY THE FIX TO EACH CHAIN INDIVIDUALLY ###
fixed_chain1 <- fix_label_switching(mu.samp1, nu.samp1, pi.samp1, X.samp1)
fixed_chain2 <- fix_label_switching(mu.samp2, nu.samp2, pi.samp2, X.samp2)

# Extract the fixed matrices
mu.fixed1 <- fixed_chain1$mu
nu.fixed1 <- fixed_chain1$nu
mu.fixed2 <- fixed_chain2$mu
nu.fixed2 <- fixed_chain2$nu
```

Trace Plots: Both Chains After Label Switching Fix

Trace plots for both chains are overlaid after fixing label switching. If the chains have converged to the same stationary distribution, the two traces should occupy the same range and mix across each other throughout the run.

```
### 3. PLOT THE OVERLAPPING CHAINS ###
par(mfrow = c(3, 1))
```

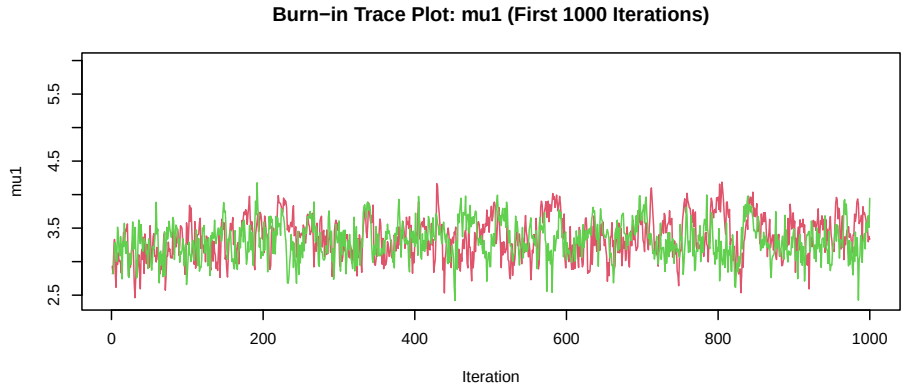
```r
# Shared y-axis limits across mu1 and mu2 panels
ymin_mu <- min(mu.fixed1[, 1], mu.fixed2[, 1], mu.fixed1[, 2], mu.fixed2[, 2])
ymax_mu <- max(mu.fixed1[, 1], mu.fixed2[, 1], mu.fixed1[, 2], mu.fixed2[, 2])

# Plot mu1 (State 1) - Red and Green overlapping
plot(1:numsamp, mu.fixed1[, 1],
    type = "l", col = 2, ylim = c(ymin_mu, ymax_mu),
    main = "Fixed Trace Plot: mu1 (State 1)", ylab = "mu1", xlab = "Iteration"
)
lines(1:numsamp, mu.fixed2[, 1], col = 3)

# Plot mu2 (State 2) - Red and Green overlapping
plot(1:numsamp, mu.fixed1[, 2],
    type = "l", col = 2, ylim = c(ymin_mu, ymax_mu),
    main = "Fixed Trace Plot: mu2 (State 2)", ylab = "mu2", xlab = "Iteration"
)
lines(1:numsamp, mu.fixed2[, 2], col = 3)

# Plot nu21 (State 2 to State 1 transition) - Red and Green overlapping
ymin <- min(nu.fixed1[, 2], nu.fixed2[, 2])
ymax <- max(nu.fixed1[, 2], nu.fixed2[, 2])
plot(1:numsamp, nu.fixed1[, 2],
    type = "l", col = 2, ylim = c(ymin, ymax),
    main = "Fixed Trace Plot: nu21 (State 2 to State 1)", ylab = "nu21", xlab = "Iteration"
)
lines(1:numsamp, nu.fixed2[, 2], col = 3)
```

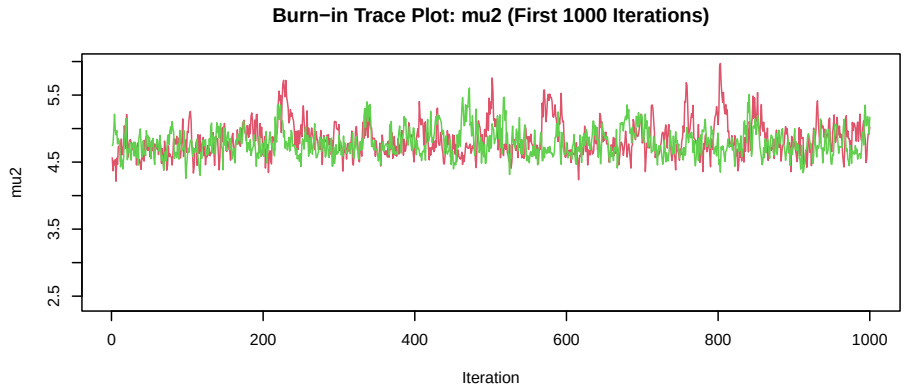
Burn-In Diagnostics

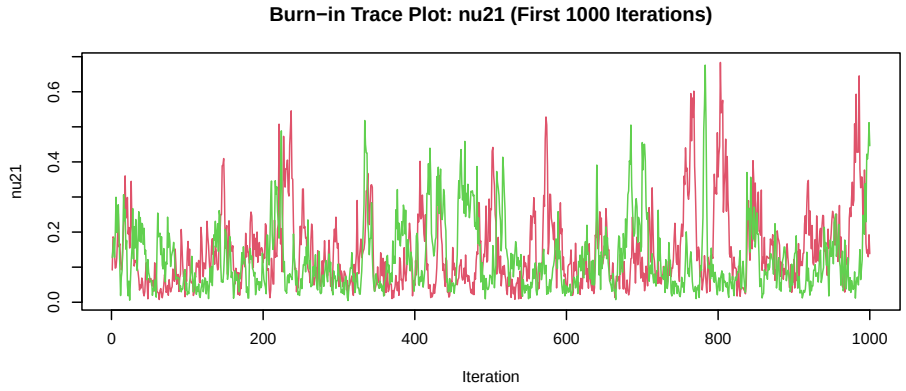
The first several hundred iterations may still reflect the starting values rather than the stationary distribution. Zooming in on the first 1,000 iterations shows how quickly each chain converges from its starting point. Samples from this burn-in window will be discarded before inference.

```
# Checking convergence during the burn-in period using FIXED chains
par(mfrow = c(3, 1))

# Calculate global y-limits for the means so the plots are aligned
ymin_mu <- min(mu.fixed1[1:1000, 1:2], mu.fixed2[1:1000, 1:2])
```

```r
ymax_mu <- max(mu.fixed1[1:1000, 1:2], mu.fixed2[1:1000, 1:2])

# Plot mu1 (State 1) - First 1000 iterations
plot(1:1000, mu.fixed1[1:1000, 1],
    type = "l", col = 2, ylim = c(ymin_mu, ymax_mu),
    main = "Burn-in Trace Plot: mu1 (First 1000 Iterations)", ylab = "mu1", xlab = "Iteration"
)
lines(1:1000, mu.fixed2[1:1000, 1], col = 3)

# Plot mu2 (State 2) - First 1000 iterations
plot(1:1000, mu.fixed1[1:1000, 2],
    type = "l", col = 2, ylim = c(ymin_mu, ymax_mu),
    main = "Burn-in Trace Plot: mu2 (First 1000 Iterations)", ylab = "mu2", xlab = "Iteration"
)
lines(1:1000, mu.fixed2[1:1000, 2], col = 3)

# Plot nu21 - First 1000 iterations
ymin_nu <- min(nu.fixed1[1:1000, 2], nu.fixed2[1:1000, 2])
ymax_nu <- max(nu.fixed1[1:1000, 2], nu.fixed2[1:1000, 2])
plot(1:1000, nu.fixed1[1:1000, 2],
    type = "l", col = 2, ylim = c(ymin_nu, ymax_nu),
    main = "Burn-in Trace Plot: nu21 (First 1000 Iterations)", ylab = "nu21", xlab = "Iteration"
)
lines(1:1000, nu.fixed2[1:1000, 2], col = 3)
```





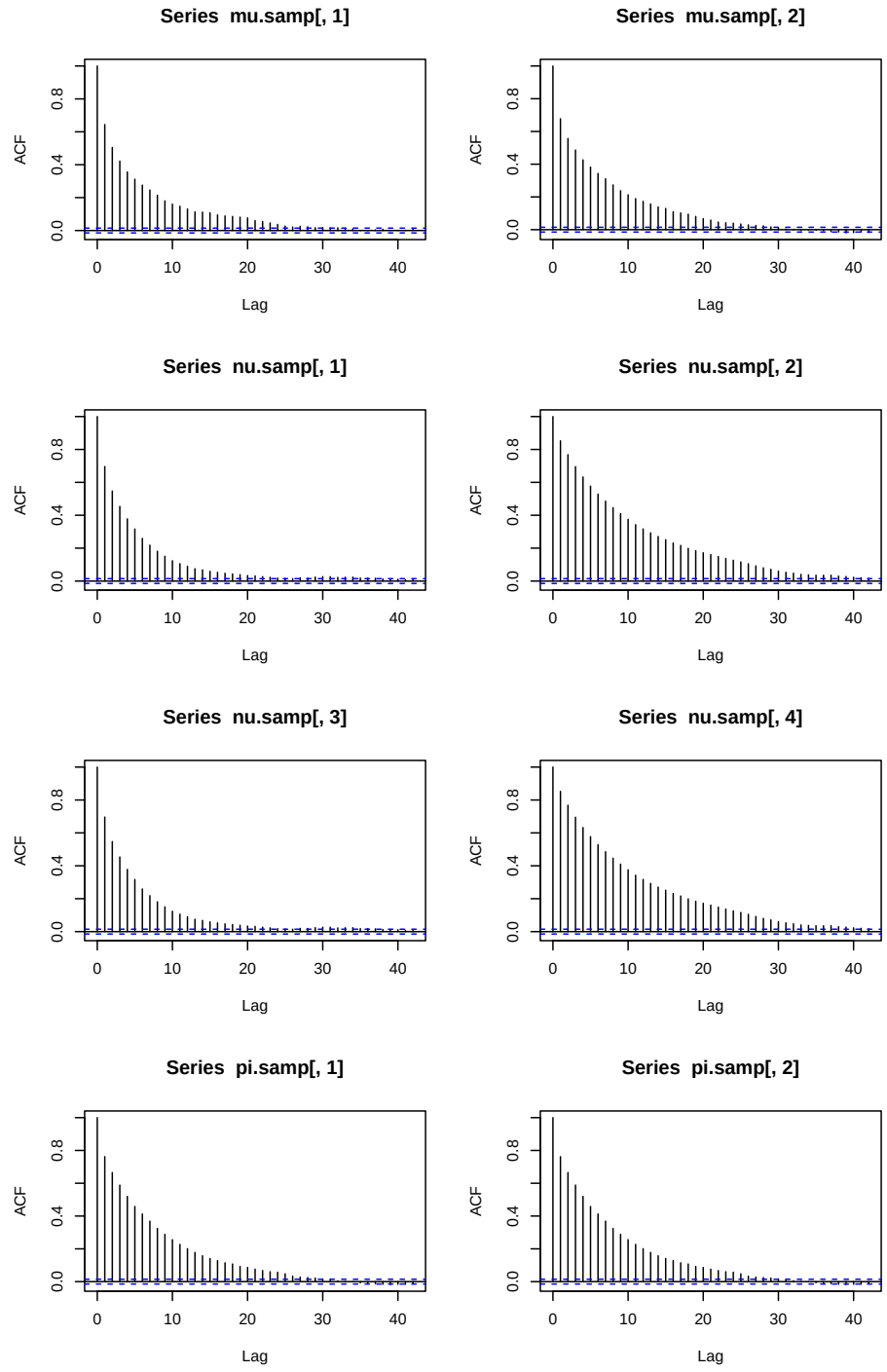


Post-Burn-In: Combining Chains, ACF, and Thinning

The first 1,000 iterations are discarded as burn-in. The two chains are then combined into a single sample pool. ACF plots are used to check autocorrelation — high autocorrelation means successive draws are nearly redundant. To reduce it, the combined chain is **thinned** by keeping only every 5th sample. ACF plots are checked again after thinning to confirm rapid decay.

```
postburn <- 1001:numsamp
X.samp <- rbind(fixed_chain1$X[postburn, ], fixed_chain2$X[postburn, ])
mu.samp <- rbind(fixed_chain1$mu[postburn, ], fixed_chain2$mu[postburn, ])
nu.samp <- rbind(fixed_chain1$nu[postburn, ], fixed_chain2$nu[postburn, ])
```

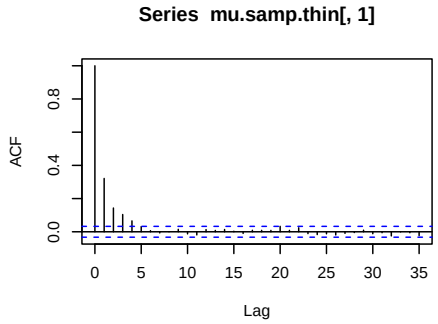
```r
pi.samp <- rbind(fixed_chain1$pi[postburn, ], fixed_chain2$pi[postburn, ])

### checking acf
par(mfrow = c(4, 2))
acf(mu.samp[, 1])
acf(mu.samp[, 2])
acf(nu.samp[, 1])
acf(nu.samp[, 2])
acf(nu.samp[, 3])
acf(nu.samp[, 4])
acf(pi.samp[, 1])
acf(pi.samp[, 2])
```

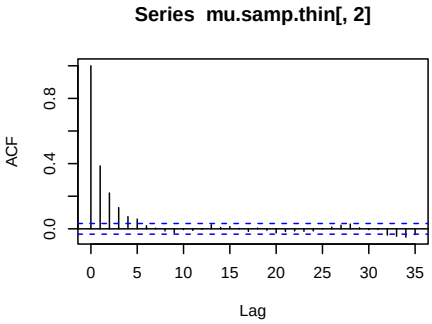


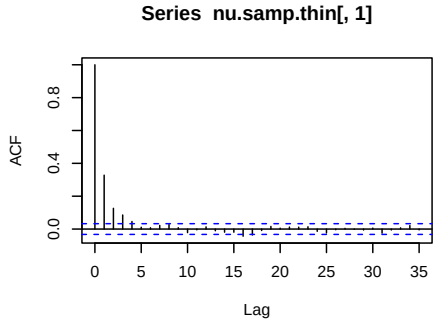
The ACF plots above show autocorrelation before thinning. If autocorrelation is high, every 5th sample is kept to reduce it. The thinned samples are stored as the final posterior draws used for inference.

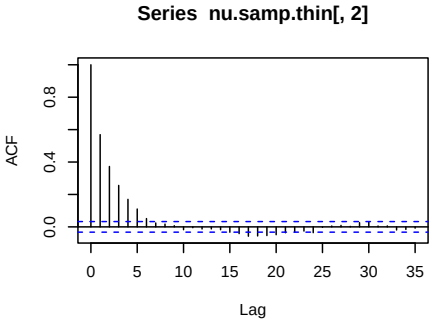
```
### thinning
temp <- 5 * (c(1:(length(mu.samp[, 1]) / 5)))

X.samp.thin <- X.samp[temp, ]
mu.samp.thin <- mu.samp[temp, ]
```

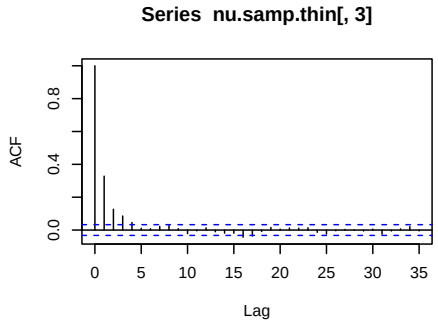
```r
nu.samp.thin <- nu.samp[temp, ]
pi.samp.thin <- pi.samp[temp, ]

### checking acf
par(mfrow = c(4, 2))
acf(mu.samp.thin[, 1])
acf(mu.samp.thin[, 2])
acf(nu.samp.thin[, 1])
acf(nu.samp.thin[, 2])
acf(nu.samp.thin[, 3])
acf(nu.samp.thin[, 4])
acf(pi.samp.thin[, 1])
acf(pi.samp.thin[, 2])
```

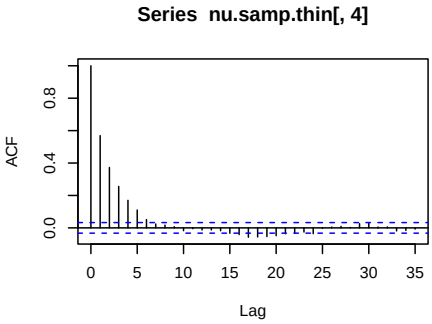


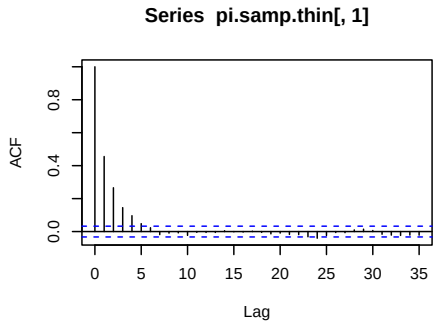


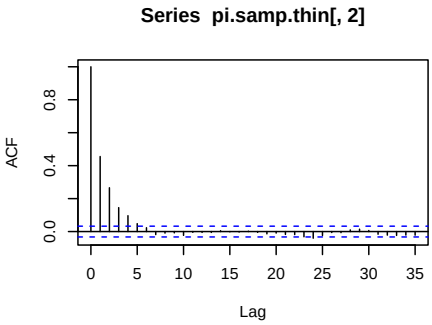










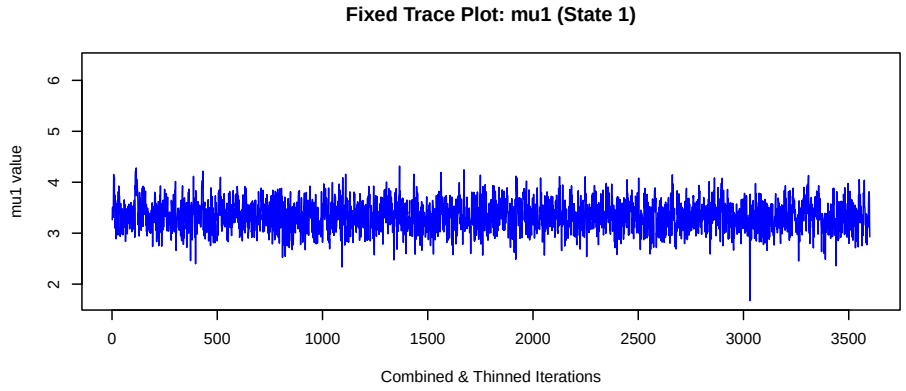


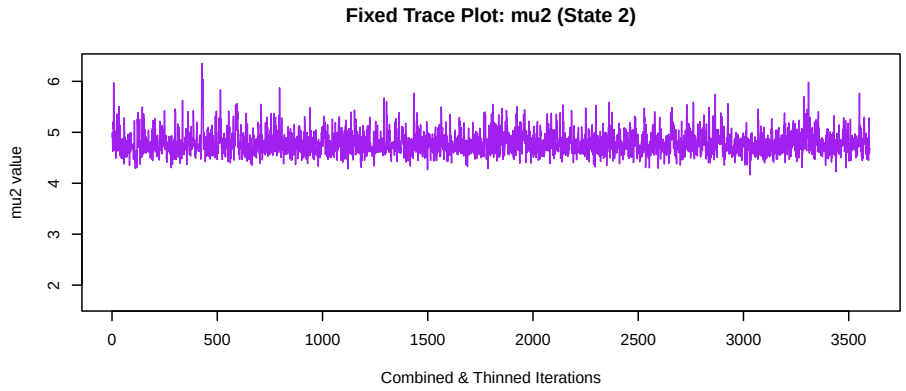

```
X.final <- X.samp.thin
mu.final <- mu.samp.thin
nu.final <- nu.samp.thin
pi.final <- pi.samp.thin
```

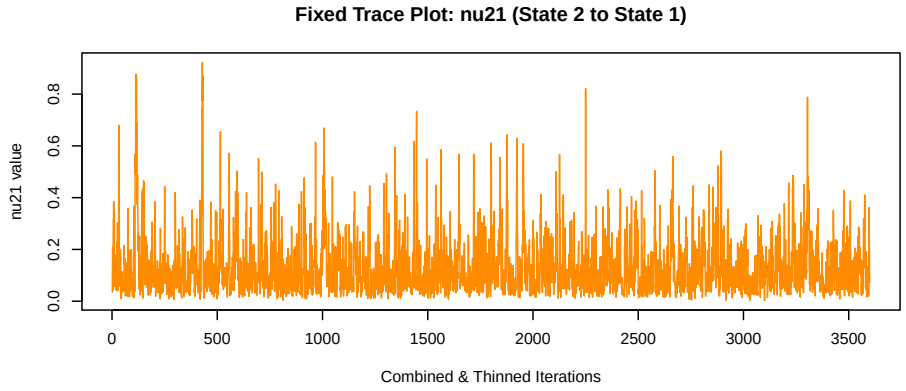
Trace Plots After Thinning

Trace plots for the thinned, combined posterior samples. These should show stable mixing with no trend or drift, confirming the chain has reached stationarity.

```r
# Checking trace plots AFTER fixing label switching (with consistent Y-axes)
par(mfrow = c(3, 1))

# 1. Find the global minimum and maximum across both mu1 and mu2
ymin <- min(mu.final[, 1], mu.final[, 2])
ymax <- max(mu.final[, 1], mu.final[, 2])

# 2. Plot State 1 Mean (with forced ylim)
plot(mu.final[, 1],
    type = "l", col = "blue",
    ylim = c(ymin, ymax),
    main = "Fixed Trace Plot: mu1 (State 1)",
    ylab = "mu1 value", xlab = "Combined & Thinned Iterations"
)

# 3. Plot State 2 Mean (with forced ylim)
plot(mu.final[, 2],
    type = "l", col = "purple",
    ylim = c(ymin, ymax),
    main = "Fixed Trace Plot: mu2 (State 2)",
    ylab = "mu2 value", xlab = "Combined & Thinned Iterations"
)

# 4. Plot Transition Probability (nu_21) - Left as is
plot(nu.final[, 2],
    type = "l", col = "darkorange",
    main = "Fixed Trace Plot: nu21 (State 2 to State 1)",
    ylab = "nu21 value", xlab = "Combined & Thinned Iterations"
)
```





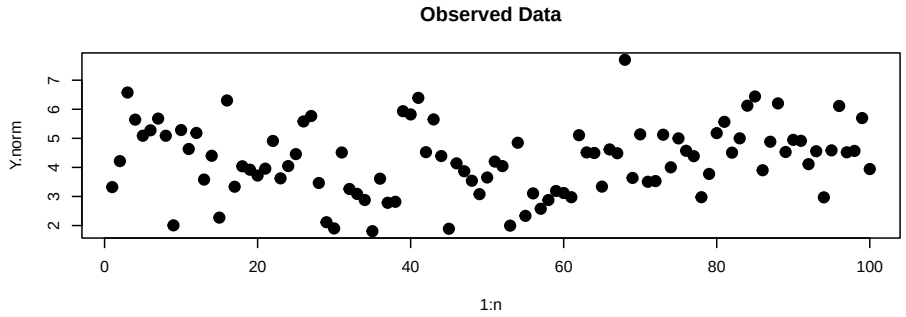


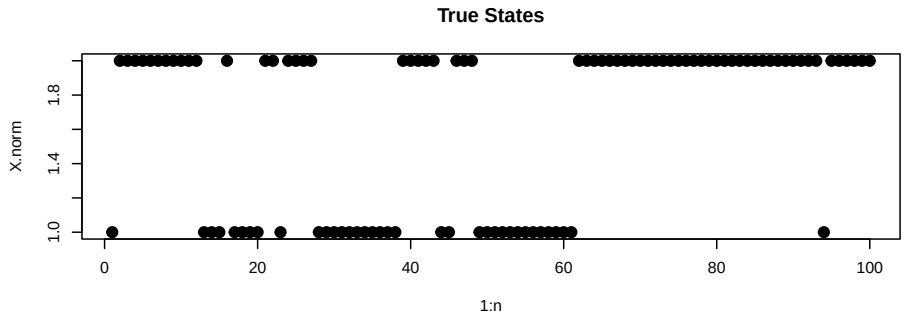
Posterior State Probabilities and Final Comparison

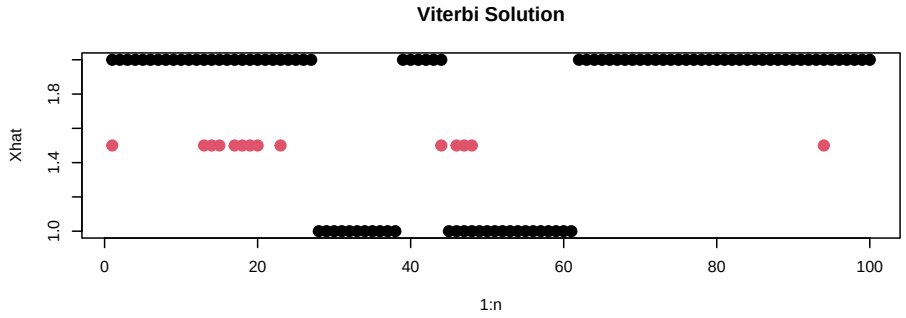
For each time point t, the posterior probability that the hidden state is 2 is computed as the fraction of posterior samples with $X_t = 2$. This gives a soft, probabilistic estimate of the state at each time point — unlike the Viterbi estimate, which gives a single hard assignment. The four panels compare the observed data, true hidden states, Viterbi solution, and the full posterior state probabilities.

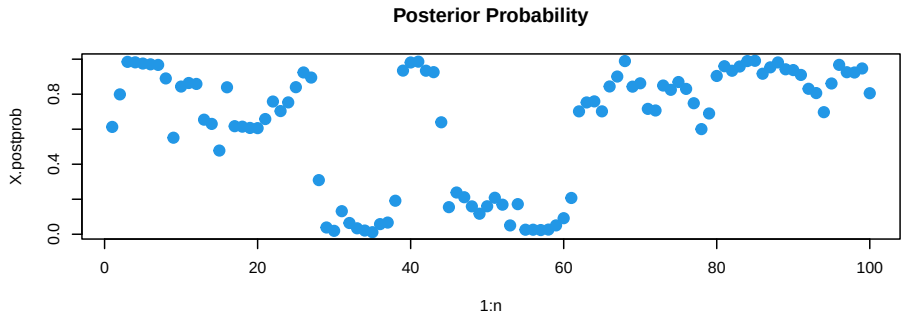
```
### calculating posterior probabilities of hidden states
X.postprob <- rep(NA, n)
for (j in 1:n) {
    X.postprob[j] <- sum(X.final[, j] == 2) / length(X.final[, 1])
```

```
}

### checking our inferred states
par(mfrow = c(4, 1))
plot(1:n, Y.norm, pch = 19, main = "Observed Data", cex = 1.5)
plot(1:n, X.norm, pch = 19, main = "True States", cex = 1.5)
plot(1:n, Xhat, pch = 19, main = "Viterbi Solution", cex = 1.5)
for (i in 1:n) {
    if (Xhat[i] != X.norm[i]) {
        points(i, 1.5, col = 2, pch = 19, cex = 1.5)
    }
}
plot(1:n, X.postprob, col = 4, pch = 19, main = "Posterior Probability", cex = 1.5)
```


Observed Data
Y.norm
2
3
4
5
6
7
0
20
40
60
80
100
1:n


True States
X.norm
1.0
1.4
1.8
0
20
40
60
80
100
1:n


Viterbi Solution
Xhat
1.0
1.4
1.8
0
20
40
60
80
100
1:n


Posterior Probability
X.postprob
0.0
0.4
0.8
0
20
40
60
80
100
1:n